EffortGraph: Joint Model and Reasoning Effort Selection with Graph Neural Networks

Chenyue Li
Stanford University
chenyuel@stanford.edu

Yanzhen Shen
Stanford University
yanzhen4@stanford.edu

1 Introduction

Choosing how much computation to spend on a query requires deciding both which LLM to use and how much reasoning it should perform. We use a graph neural network (GNN) to make this joint decision for cost-aware question answering. Our router represents models, reasoning efforts, queries, and tasks in a heterogeneous graph, allowing related configurations to share information from past responses. We ask whether this shared structure yields better quality–cost tradeoffs than fixed configurations and non-graph routing.

Related work. Prior work approaches efficient LLM use through cost-aware cascades, as in FrugalGPT [7], and preference-based strong/weak model selection, as in RouteLLM [8]. RouterDC [9] learns query–model compatibility with dual contrastive losses, while EmbedLLM [10] learns reusable model representations. RouterBench [11] provides a framework for evaluating routing quality and cost. GraphRouter [1] connects tasks, queries, and models for LLM selection. We additionally represent reasoning effort as nodes to share information across effort settings.

Model configurations. We consider three GPT-5.4 models, each with low, medium, and high reasoning effort, giving nine candidate configurations (Table 1) [6]. Using one model generation lets us study differences in model size and effort within a manageable candidate set. Before collection, we will verify endpoint support and fix model snapshots and prices.

Table 1: Candidate models and reasoning effort settings.

Model ID	Reasoning efforts
`gpt-5.4-nano`	low, medium, high
`gpt-5.4-mini`	low, medium, high
`gpt-5.4`	low, medium, high

2 Datasets and experimental setting

We use three public benchmarks to cover general knowledge, specialist knowledge, and automatically graded reasoning tasks. MMLU-Pro [2] contains 12,032 test questions across 14 subjects, while SuperGPQA [3] contains 26,529 graduate-level questions across 72 fields; we score both by exact-match answer accuracy. LiveBench [4] broadens this coverage with Mathematics, Reasoning, Coding, Data Analysis, and Instruction Following. After filtering removed questions and deduplicating prompts, our selected public snapshot contains 232, 150, 128, 149, and 200 questions in these categories, respectively. We evaluate LiveBench responses with its native task-specific scoring functions, including code execution for Coding, after validating the graders. Because these public questions predate the candidate models, we cannot rule out training-data contamination.

CS224W, Stanford University. Project proposal draft.

To represent variation within each benchmark, we select five task groups per dataset: five MMLU-Pro subjects, five SuperGPQA fields, and the five LiveBench categories above. The MMLU-Pro subjects and SuperGPQA fields will be fixed before collection, with at least 120 eligible questions each. LiveBench categories provide enough questions for this design, whereas individual subtasks are too small; we stratify within categories where feasible. These groups define 15 dataset-specific task nodes, with labels assumed available at inference.

We sample 120 eligible questions per group and split them into 96 training and 24 test questions, with no validation set. Each dataset therefore contributes 480 training and 120 test questions, for a total of 1,440 training and 360 test questions. All responses to the same question remain in one split to prevent leakage. The test set is reserved for final evaluation. Collecting responses from all nine configurations requires 16,200 model calls, excluding retries. We will make the resulting routing data and splits public before submission, subject to source licenses.

3 Proposed method

The benchmarks group questions by subject or capability, while each query is evaluated under the same nine configurations. We represent these shared relationships in a heterogeneous graph:

$$\boxed{\text{Effort}} \longleftrightarrow \boxed{\text{Model}} \longleftrightarrow \boxed{\text{Query}} \longleftrightarrow \boxed{\text{Task}}. \tag{1}$$

Each query is linked to its task and to models with observed responses, while each model is linked to the effort settings it supports. For a response generated by model m at effort e on query q, we retain a separate query–model edge indexed by (q, m, e), carrying quality, cost, and an effort embedding. Bidirectional, relation-specific attention combines task context with historical performance while preserving effort-specific outcomes.

We implement the graph in PyG using `HeteroData` and relation-specific graph attention layers [5]. We initialize nodes with text embeddings and use two GAT layers with hidden size 32. The resulting representations are combined to score each valid configuration:

$$s_\theta(q, m, e) = \mathrm{MLP}_\theta([h_q \| h_{t(q)} \| h_m \| h_e]), \qquad (m^*, e^*) = \arg\max_{(m,e)\in\mathcal{C}} s_\theta(q, m, e), \tag{2}$$

where $t(q)$ is the query's task and $\mathcal{C}$ is the set of supported model–effort pairs. We train the scorer with a binary classification objective, labeling the configuration with the highest quality–cost reward as positive:

$$r(q, m, e) = \alpha\widetilde{p}(q, m, e) - \beta\widetilde{c}(q, m, e), \qquad y_{q,m,e} = \mathbf{1}[(m, e) = \arg\max_{(m',e')\in\mathcal{C}} r(q, m', e')]. \tag{3}$$

Here p is the benchmark score and c is the cost of billed tokens, including reasoning tokens, at the recorded prices. We normalize both using training data and resolve ties deterministically. We consider $(\alpha, \beta) \in \{(1, 0), (0.5, 0.5), (0.2, 0.8)\}$ to study different quality–cost preferences. Training uses Adam with learning rate 10^{-3} and batch size 32 for 1,000 epochs, retaining the final checkpoint.

To prevent target leakage, we mask all response edges for the query being predicted and exclude test outcomes from message passing. At inference, the router selects a configuration using only the query and its task label.

4 Evaluation and expected contribution

We compare our router with each fixed model–effort configuration and a non-graph MLP using identical query/task features and separate model and effort embeddings. This comparison isolates the benefit of message passing beyond shared input features and embeddings. Both learned routers use the same splits, responses, supervision, and training protocol. We separately remove effort nodes while retaining effort identifiers in edge features and the scoring head, and replace within-dataset task nodes with dataset-level nodes. These ablations test whether explicit effort connections and finer task structure improve routing. We report per-dataset benchmark scores, macro-averaged normalized quality, cost per query, routing overhead, and quality–cost curves, with uncertainty estimated across seeds and paired query resampling.

References

[1] Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections. ICLR, 2025. https://arxiv.org/abs/2410.03834.

[2] Yubo Wang et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. NeurIPS Datasets and Benchmarks, 2024. https://arxiv.org/abs/2406.01574.

[3] SuperGPQA Team. SuperGPQA: Scaling LLM Evaluation across 285 Graduate Disciplines. 2025. https://arxiv.org/abs/2502.14739. Data: https://github.com/SuperGPQA/SuperGPQA.

[4] Colin White et al. LiveBench: A Challenging, Contamination-Free LLM Benchmark. ICLR, 2025. https://arxiv.org/abs/2406.19314. Evaluation: https://github.com/LiveBench/LiveBench.

[5] Petar Veličković et al. Graph Attention Networks. ICLR, 2018. https://arxiv.org/abs/1710.10903.

[6] OpenAI. API model catalog and model-specific specifications. Accessed October 17, 2026. https://developers.openai.com/api/docs/models. Reasoning: https://developers.openai.com/api/docs/guides/reasoning.

[7] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. TMLR, 2024. https://arxiv.org/abs/2305.05176.

[8] Isaac Ong et al. RouteLLM: Learning to Route LLMs from Preference Data. ICLR, 2025. https://arxiv.org/abs/2406.18665.

[9] Shuhao Chen et al. RouterDC: Query-Based Router by Dual Contrastive Learning for Assembling Large Language Models. NeurIPS, 2024. https://arxiv.org/abs/2409.19886.

[10] Richard Zhuang et al. EmbedLLM: Learning Compact Representations of Large Language Models. ICLR, 2025. https://arxiv.org/abs/2410.02223.

[11] Qitian Jason Hu et al. RouterBench: A Benchmark for Multi-LLM Routing System. 2024. https://arxiv.org/abs/2403.12031.